

**LAPORAN TUGAS INDIVIDU : EKSPERIMEN OPERASI DASAR PADA SINYAL DAN
CITRA**

PENGOLAHAN SINYAL DIGITAL

Dosen Pengampu : Al – Ustadz Assoc. Prof. Dr. Ir. Oddy Virgantara Putra, S.Kom., M.T., IPP.



Disusun Oleh :

Zharifah Zahra Salsabila

452024618106

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS DARUSSALAM GONTOR

2026/1447

1. PENDAHULUAN

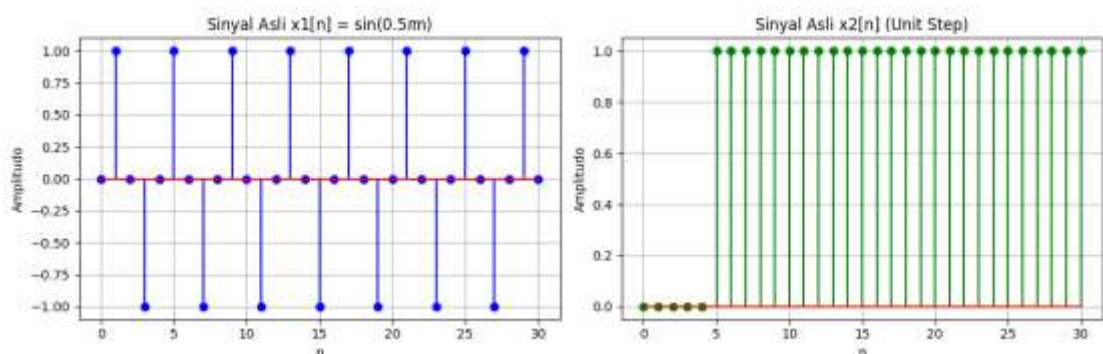
Eksperimen ini bertujuan untuk mengimplementasikan dan menganalisis operasi dasar pada sinyal 1D dan citra 2D menggunakan bahasa pemrograman Python. Operasi dasar yang dieksplorasi meliputi penjumlahan, penggeseran (translasi), dan amplifikasi. Selain itu, laporan ini juga menguji sifat linieritas suatu sistem melalui prinsip homogenitas dan additivitas, serta menghubungkan konsep matematis tersebut dengan aplikasi di dunia nyata seperti *image blending*.

2. OPERASI SINYAL 1D

A. Membuat Sinyal Diskrit

```
import numpy as np
import matplotlib.pyplot as plt
# 1. Menentukan rentang nilai n (0 sampai 30)
n = np.arange(0, 31)
# 2. Membuat Sinyal pertama:  $x_1[n] = \sin(0.5 * \pi * n)$ 
x1 = np.sin(0.5 * np.pi * n)
# 3. Membuat Sinyal kedua: unit step (0 jika  $n < 5$ , 1 jika  $n \geq 5$ )
x2 = np.where(n >= 5, 1.0, 0.0)
# 4. Visualisasi Plot A.1 (Menampilkan  $x_1$  dan  $x_2$ )
plt.figure(figsize=(12, 4))
# Plot  $x_1[n]$ 
plt.subplot(1, 2, 1)
plt.stem(n, x1, linefmt='b-', markerfmt='bo', basefmt='r-')
plt.title('Sinyal Asli  $x_1[n] = \sin(0.5\pi n)$ ')
plt.xlabel('n')
plt.ylabel('Amplitudo')
plt.grid(True)
# Plot  $x_2[n]$ 
plt.subplot(1, 2, 2)
plt.stem(n, x2, linefmt='g-', markerfmt='go', basefmt='r-')
plt.title('Sinyal Asli  $x_2[n]$  (Unit Step)')
plt.xlabel('n')
plt.ylabel('Amplitudo')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output :

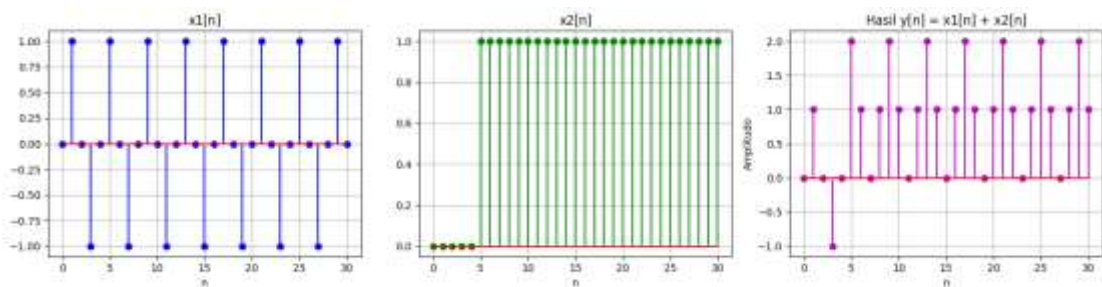


B. Operasi Penjumlahan Sinyal

```
# 1. Operasi Penjumlahan Sinyal
y_add = x1 + x2
# 2. Visualisasi Plot A.2 (Menampilkan x1, x2, dan y)
plt.figure(figsize=(15, 4))
# Plot x1[n]
plt.subplot(1, 3, 1)
plt.stem(n, x1, linefmt='b-', markerfmt='bo', basefmt='r-')
plt.title('x1[n]')
plt.xlabel('n')
plt.grid(True)
# Plot x2[n]
plt.subplot(1, 3, 2)
plt.stem(n, x2, linefmt='g-', markerfmt='go', basefmt='r-')
plt.title('x2[n]')
plt.xlabel('n')
plt.grid(True)
# Plot hasil y[n]
plt.subplot(1, 3, 3)
plt.stem(n, y_add, linefmt='m-', markerfmt='mo', basefmt='r-')
plt.title('Hasil y[n] = x1[n] + x2[n]')
plt.xlabel('n')
plt.ylabel('Amplitudo')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Output :



Jawaban pertanyaan analisis :

- 1) Amplitudo sinyal hasil penjumlahan pada setiap titik waktu (n) merupakan hasil penjumlahan langsung dari amplitudo kedua sinyal asal pada titik waktu yang sama secara *element-wise* ($y[n] = x_1[n] + x_2[n]$). Jika kedua sinyal bernilai positif, amplitudonya akan bertambah besar (saling menguatkan). Jika salah satu bernilai negatif, akan saling mengurangi.
- 2) Bentuknya menjadi kombinasi dari keduanya. Dalam kasus eksperimenmu (gelombang sinus ditambah *unit step* bernilai 1 mulai $n \geq 5$), gelombang hasil akan tetap memiliki bentuk

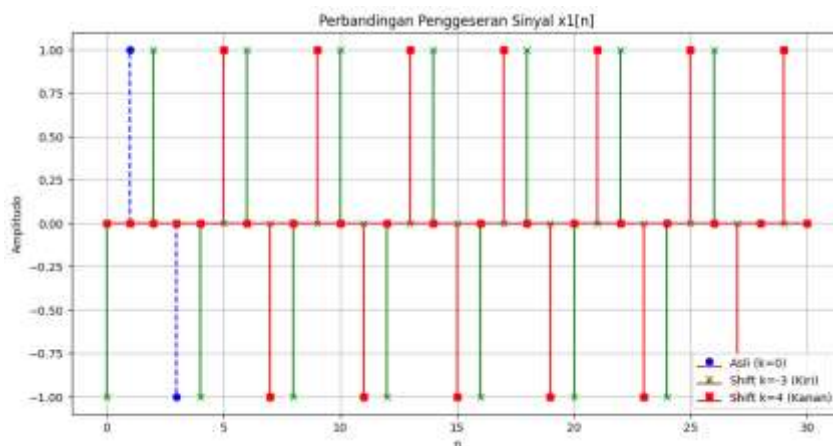
osilasi seperti sinyal sinus (sinyal asal 1), namun akan mengalami kenaikan/pergeseran titik tengah (offset DC) sebesar 1 pada area di mana *unit step* aktif.

- 3) Dapat digunakan sebagai Audio Mixing, Noise Cancellation dan Sensor Fusion

C. Operasi Penggeseran Sinyal

```
def shift_signal(x, k):
    if k >= 0:
        return np.concatenate((np.zeros(k), x[:len(x)-k]))
    else:
        return np.concatenate((x[-k:], np.zeros(-k)))
# Lakukan penggeseran pada sinyal x1[n]
y_shift_neg3 = shift_signal(x1, -3)
y_shift_0 = shift_signal(x1, 0)
y_shift_pos4 = shift_signal(x1, 4)
# Plot perbandingan dalam satu gambar [cite: 68]
plt.figure(figsize=(12, 6))
plt.stem(n, x1, linefmt='b--', markerfmt='bo', label='Asli (k=0)')
plt.stem(n, y_shift_neg3, linefmt='g-', markerfmt='gx', label='Shift k=-3 (Kiri)')
plt.stem(n, y_shift_pos4, linefmt='r-', markerfmt='rs', label='Shift k=4 (Kanan)')
plt.title('Perbandingan Penggeseran Sinyal x1[n]')
plt.xlabel('n')
plt.ylabel('Amplitudo')
plt.legend()
plt.grid(True)
plt.show()
```

Output :



Jawaban pertanyaan analisis :

- 1) Perbedaan antara k positif dan k negatif :

- Jika k bernilai positif ($k > 0$): Sinyal akan mengalami penundaan (*delay*) dan grafiknya akan bergeser ke kanan. Misalnya pada $y[n] = x[n-3]$, titik yang awalnya berada di waktu $n=0$ akan mundur dan baru muncul pada waktu $n=3$.
- Jika k bernilai negatif ($k < 0$): Sinyal akan mengalami percepatan (*advance*) dan grafiknya akan bergeser ke kiri. Misalnya pada $y[n] = x[n+3]$ (karena dikurangi nilai negatif), titik yang tadinya di $n=3$ akan ditarik maju dan sudah muncul sejak waktu $n=0$.

- 2) Dalam pengolahan sinyal diskrit, indeks n mewakili urutan waktu (sampel). Untuk mensimulasikan sebuah *delay* (misalnya suara gema/pantulan atau penundaan transmisi jaringan), kita cukup menggunakan nilai k positif. Secara komputasi, array data sinyal akan digeser posisinya sejauh k langkah ke kanan. Hal ini secara akurat meniru kondisi di dunia nyata di mana suatu sinyal atau suara tiba lebih lambat dari sumber aslinya.
- 3) *Time alignment* (penyelarasan waktu) sangat krusial ketika kita harus memproses atau menggabungkan dua sinyal atau lebih (seperti eksperimen penjumlahan sinyal di Bagian A.2). Jika sinyal-sinyal tersebut tidak diselaraskan waktunya terlebih dahulu, operasi penggabungannya akan kacau.

D. Operasi Amplifikasi Sinyal

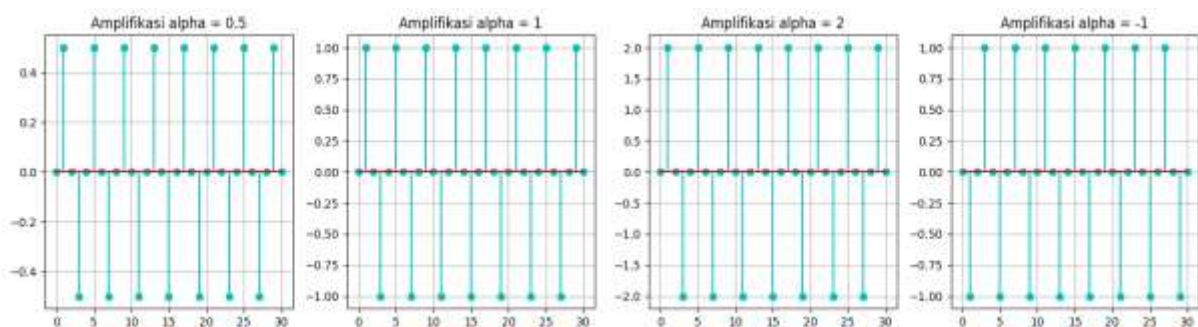
```

alphas = [0.5, 1, 2, -1]
plt.figure(figsize=(15, 4))
for i, alpha in enumerate(alphas):
    y_amp = alpha * x1 # Operasi amplifikasi [cite: 77]
    plt.subplot(1, 4, i+1)
    plt.stem(n, y_amp, linefmt='c-', basefmt='r-')
    plt.title(f'Amplifikasi alpha = {alpha}')
    plt.grid(True)

plt.tight_layout()
plt.show()

```

Output :



Efek pada Sinyal :

Nilai α	Efek terhadap Sinyal
0.5	Sinyal mengalami pelemahan (attenuation). Amplitudo menyusut menjadi setengah dari ukuran aslinya.

1	Tidak ada perubahan pada sinyal (bertindak sebagai nilai identitas).
2	Sinyal mengalami penguatan (<i>amplification</i>). Amplitudo membesar menjadi dua kali lipat dari aslinya.
-1	Sinyal mengalami pembalikan fasa (<i>phase inversion</i>). Amplitudonya tetap sama, namun posisinya dicerminkan secara vertikal (yang positif jadi negatif, dan sebaliknya).

Jawaban pertanyaan analisis :

- 1) Sinyal mengalami penguatan atau perbesaran amplitudo. Puncak gelombang akan menjadi semakin tinggi dan lembahnya akan menjadi semakin dalam secara proporsional.
- 2) Sinyal mengalami pelemahan (*attenuation*). Amplitudonya menjadi lebih kecil dan grafik sinyal akan terlihat lebih mendatar atau merata dibandingkan sinyal aslinya.
- 3) Sinyal mengalami pembalikan fasa (*phase inversion*). Bagian gelombang yang tadinya berada di area positif (di atas sumbu-x) akan terbalik menjadi negatif (di bawah sumbu-x), dan sebaliknya. Besaran pembesaran atau penyusutannya bergantung pada nilai absolut dari α tersebut.
- 4) Konsep matematis amplifikasi (α) ini sama persis dengan fungsi kenop *Volume* atau *Gain* pada *amplifier* audio.
 - Ketika kita menaikkan *gain* ($\alpha > 1$), suara menjadi lebih keras karena amplitudonya membesar.
 - Ketika kita menurunkannya ($0 < \alpha < 1$), suara menjadi lebih pelan.
 - Jika faktornya disetel menjadi negatif, itu sama seperti menekan tombol *Phase Invert* pada konsol *mixer* audio, yang biasa digunakan oleh *sound engineer* untuk mencegah masalah *feedback* atau *phase cancellation* (suara saling menghilangkan).

3. Operasi pada Citra 2D

A. Membaca dan Menampilkan Citra

```
import cv2
import matplotlib.pyplot as plt
img1 = cv2.imread('/img1.jpg', cv2.IMREAD_GRAYSCALE)
print("--- Properti Citra Asli ---")
print(f"Ukuran Citra (Resolusi): {img1.shape}")
print(f"Type Data Citra: {img1.dtype}")
print(f"Nilai Minimum Pixel: {img1.min()}")
print(f"Nilai Maksimum Pixel: {img1.max()}")

plt.imshow(img1, cmap='gray')
plt.title('Citra Asli 1')
plt.axis('off')
plt.show()
```

Output :

```
--- Properti Citra Asli ---  
Ukuran Citra (Resolusi): (580, 464)  
Tipe Data Citra: uint8  
Nilai Minimum Pixel: 0  
Nilai Maksimum Pixel: 237
```

Citra Asli 1



B. Operasi Penjumlahan Citra (Resizing)

```
img2 = cv2.imread('/img2.jpg', cv2.IMREAD_GRAYSCALE)  
  
# Proses Resizing  
if img1.shape != img2.shape:  
    print(f"Ukuran berbeda! Resizing Citra 2 dari {img2.shape}  
    menjadi {img1.shape}")  
    img2_resized = cv2.resize(img2, (img1.shape[1],  
    img1.shape[0]))  
else:  
    img2_resized = img2  
img_sum_clipped = np.clip(img1.astype(np.int32) +  
img2_resized.astype(np.int32), 0, 255).astype(np.uint8)  
  
plt.figure(figsize=(12, 4))  
plt.subplot(1, 3, 1); plt.imshow(img1, cmap='gray');  
plt.title('Citra 1'); plt.axis('off')  
plt.subplot(1, 3, 2); plt.imshow(img2_resized, cmap='gray');  
plt.title('Citra 2 (Resized)'); plt.axis('off')  
plt.subplot(1, 3, 3); plt.imshow(img_sum_clipped, cmap='gray');  
plt.title('Hasil Penjumlahan'); plt.axis('off')  
plt.show()  
Output :
```

Ukuran berbeda! Resizing Citra 2 dari (383, 680) menjadi (580, 464)



Jawaban pertanyaan analisis :

- 1) Secara matematis, citra digital direpresentasikan sebagai matriks dua dimensi (atau tiga dimensi untuk citra berwarna). Operasi penjumlahan citra dilakukan secara *element-wise*, artinya komputer menjumlahkan nilai piksel dari gambar pertama dengan nilai piksel dari gambar kedua pada koordinat baris dan kolom yang persis sama. Berdasarkan aturan dasar aljabar linier, dua buah matriks hanya bisa dijumlahkan jika ordonya (resolusi/ukurannya) sama persis.
- 2) Pada format citra standar 8-bit (tipe data uint8), nilai maksimal yang bisa disimpan oleh satu piksel adalah 255. Jika hasil penjumlahan melebihi angka ini (misalnya piksel bernilai 150 ditambah 150 menjadi 300), maka akan terjadi *overflow*. Jika tidak ditangani, komputer biasanya akan melakukan operasi *modulo* (memutar kembali nilainya dari 0), sehingga piksel yang seharusnya sangat terang (300) malah berubah menjadi gelap ($300 - 256 = 44$). Hal ini akan merusak warna dan bentuk visual gambar.
- 3) ☐ Clipping: Memotong nilai secara paksa. Semua nilai yang lebih dari 255 akan dipaksa menjadi tepat 255. Hasilnya, area yang sangat terang pada gambar akan menjadi putih polos murni (*washed out*), sehingga tekstur atau detail kecil di area terang tersebut akan hilang sama sekali.
- ☐ Normalisasi: Menyesuaikan atau memampatkan seluruh rentang nilai hasil penjumlahan secara proporsional agar kembali masuk ke rentang 0-255. Hasilnya, transisi warna tetap halus dan tidak ada detail gambar yang hilang secara tiba-tiba, meskipun tingkat kecerahan keseluruhan gambar mungkin akan sedikit meredup.
- 4) Beberapa contoh kegunaannya di dunia nyata adalah:
 - Image Blending / Double Exposure: Menggabungkan dua gambar artistik menjadi satu kesatuan (seperti gambar siluet manusia yang di dalamnya ada gambar pemandangan alam).
 - Watermarking: Menambahkan logo transparan atau teks hak cipta di atas gambar asli.
 - Noise Reduction: Pada astrofotografi atau medis, memotret objek statis berulang kali lalu menjumlahkan dan merata-ratakannya untuk menghilangkan *noise* acak dari sensor kamera.

C. Operasi Penggeseran Citra

```
def translate_image(image, dx, dy):  
    rows, cols = image.shape  
    M = np.float32([[1, 0, dx], [0, 1, dy]])  
    return cv2.warpAffine(image, M, (cols, rows))  
  
img_shift1 = translate_image(img1, 30, 0)  
img_shift2 = translate_image(img1, 0, 30)  
img_shift3 = translate_image(img1, 30, 30)  
  
plt.figure(figsize=(15, 5))  
plt.subplot(1, 4, 1); plt.imshow(img1, cmap='gray');  
plt.title('Asli'); plt.axis('off')  
plt.subplot(1, 4, 2); plt.imshow(img_shift1, cmap='gray');  
plt.title('Shift (30, 0)'); plt.axis('off')  
plt.subplot(1, 4, 3); plt.imshow(img_shift2, cmap='gray');  
plt.title('Shift (0, 30)'); plt.axis('off')  
plt.subplot(1, 4, 4); plt.imshow(img_shift3, cmap='gray');  
plt.title('Shift (30, 30)'); plt.axis('off')  
plt.show()
```

Output :



Jawaban pertanyaan analisis :

- 1) Penggeseran (translasi) mengubah koordinat spasial piksel pada citra. Efeknya, objek di dalam gambar akan berpindah posisi secara fisik ke arah horizontal (kiri/kanan pada sumbu-x) atau vertikal (atas/bawah pada sumbu-y) sesuai dengan nilai pergeseran yang diberikan, tanpa mengubah ukuran, orientasi, maupun bentuk asli dari objek tersebut.
- 2) Dalam pelatihan model kecerdasan buatan (*Machine Learning/Computer Vision*), sebuah model harus bisa mengenali suatu objek terlepas dari di mana letak objek tersebut di dalam *frame* kamera (sifat ini disebut *spatial invariance*). Translasi digunakan untuk menciptakan banyak variasi gambar baru dari satu gambar asli dengan posisi objek yang digeser-geser. Hal ini membuat model belajar lebih baik dan lebih tangguh (*robust*), sehingga model tidak hanya menghafal bahwa "objek X selalu berada di tengah gambar".
- 3) Risiko utamanya adalah kehilangan informasi visual. Jika nilai penggeseran terlalu besar, objek utama di dalam gambar bisa terpotong atau bahkan keluar sepenuhnya

dari batas dimensi matriks citra (*out of bounds*). Selain itu, pergeseran yang ekstrem akan meninggalkan area kosong yang sangat luas di sisi sebaliknya (biasanya diisi secara otomatis dengan piksel hitam bernilai 0), sehingga citra tersebut kehilangan makna visualnya dan menjadi tidak berguna untuk dianalisis lebih lanjut.

D. Operasi Amplifikasi Citra & Histogram

```
img_amp = np.clip(a * img1.astype(np.float32), 0,
255).astype(np.uint8)

# Plot Citra
plt.subplot(2, 4, i+1)
plt.imshow(img_amp, cmap='gray')
plt.title(f'Citra alpha = {a}')
plt.axis('off')

# Plot Histogram
plt.subplot(2, 4, i+5)
plt.hist(img_amp.ravel(), bins=256, range=(0, 256),
color='gray')
plt.title(f'Histogram alpha = {a}')
```

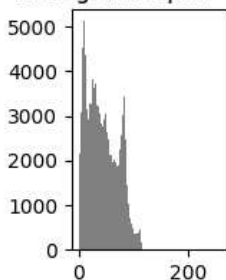
Output :

Text(0.5, 1.0, 'Histogram alpha = 0.5')

Citra alpha = 0.5



Histogram alpha = 0.5



Jawaban pertanyaan analisis :

- 1) Efek perubahan nilai α terhadap kecerahan (*brightness*) citra secara visual :
 - Ketika $\alpha > 1$ (misal $\alpha = 2$): Citra secara visual menjadi jauh lebih terang (*overexposed*). Hal ini karena nilai intensitas setiap piksel dikalikan dengan faktor yang besar, membuat nilainya mendekati atau mencapai batas maksimum (255).
 - Ketika $0 < \alpha < 1$ (misal $\alpha = 0.5$): Citra secara visual menjadi lebih gelap dan redup. Nilai intensitas setiap piksel menyusut menjadi setengahnya, sehingga gambar kehilangan energinya.

- Ketika α bernilai negatif (misal $\alpha = -1$): Karena pada kodemu menggunakan perintah `np.clip(..., 0, 255)`, mengalikan matriks dengan angka negatif akan menghasilkan nilai di bawah 0. Akibatnya, setelah di-clip, citra akan berubah menjadi hitam polos murni (nilai piksel semuanya menjadi 0).
- 2) Bentuk histogram berubah ketika $\alpha > 1$ dan ketika $0 < \alpha < 1$:
- Ketika $\alpha > 1$: Grafik histogram akan meregang (*stretch*) dan bergeser ke arah kanan (menuju nilai 255). Karena banyak nilai piksel yang melebihi 255 dan terkena fungsi *clipping*, akan terlihat lonjakan garis vertikal yang sangat tinggi di ujung paling kanan histogram.
 - Ketika $0 < \alpha < 1$: Grafik histogram akan menyusut/merapat (*compress*) dan bergeser ke arah kiri (menuju nilai 0). Distribusi warna piksel berkumpul di area intensitas rendah (gelap), sehingga grafik histogram terlihat menumpuk di sisi kiri.
- 3) Risiko utamanya adalah terjadinya Saturasi Citra (*Whiteout*). Jika nilai α terlalu besar, sebagian besar piksel (bukan cuma yang awalnya terang, tapi yang buram pun) akan dipaksa mentok di angka 255 setelah proses *clipping*. Efeknya, detail, tekstur, dan informasi penting pada area-area citra tersebut akan hilang total dan hanya menyisakan warna putih polos yang silau.
- 4) Di dunia nyata, operasi perkalian skalar ini menjadi dasar dari teknologi:
- Digital Brightness & Contrast Control: Pengaturan kecerahan dan kontras pada layar HP, monitor, atau aplikasi edit foto seperti Photoshop dan Lightroom.
 - Enhancement Citra Medis: Memperjelas detail gambar yang buram pada hasil rontgen (Sinar-X), CT Scan, atau MRI agar dokter bisa mendiagnosis penyakit dengan lebih akurat.
 - Night Vision & Surveillance: Memperkuat sinyal cahaya yang sangat lemah pada kamera pengawas (CCTV) malam hari agar objek atau orang tetap dapat terlihat jelas di dalam kegelapan.

4. BAGIAN C : Uji Sistem Linier

A. Uji Homogenitas

```
import numpy as np
import matplotlib.pyplot as plt

def T(x):
    return 2 * x
n = np.arange(0, 31)
x_n = np.sin(0.5 * np.pi * n)

alphas = [0.5, 2, -1]

plt.figure(figsize=(15, 10))

for i, alpha in enumerate(alphas):
    left_side = T(alpha * x_n)
    right_side = alpha * T(x_n)
    plt.subplot(3, 2, 2*i + 1)
```

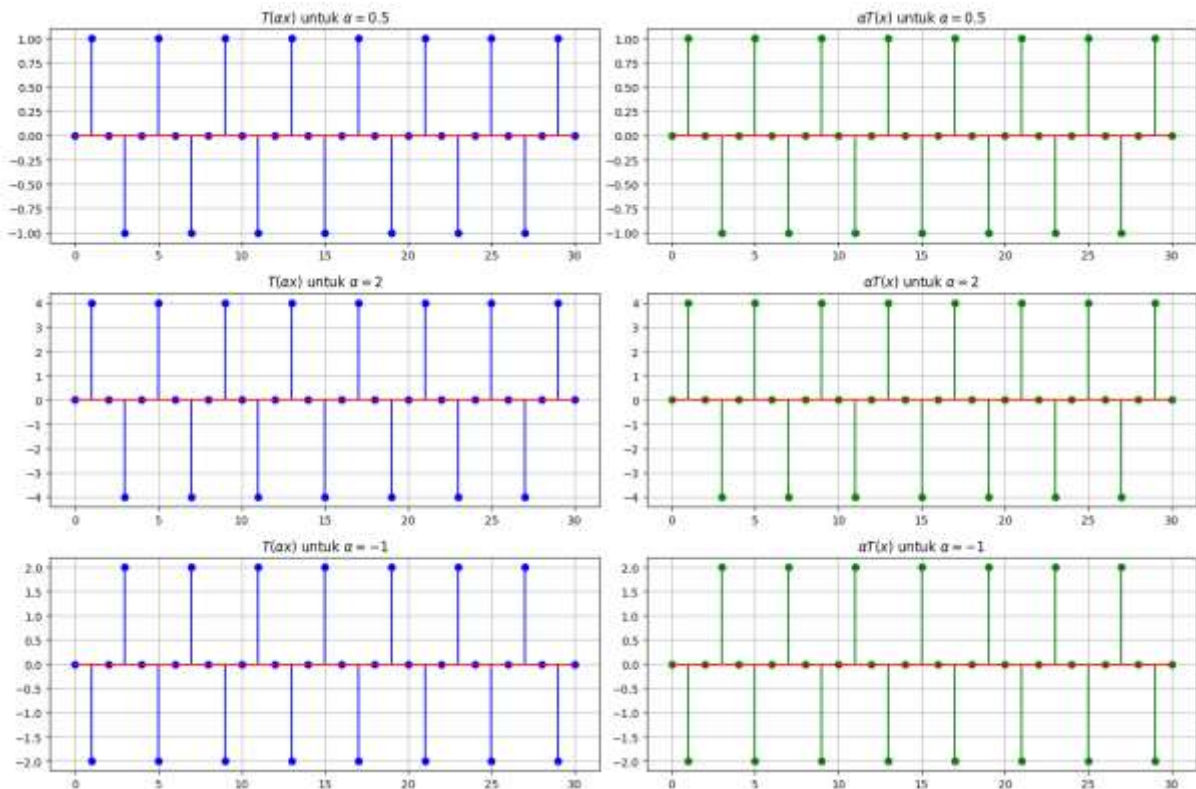
```

plt.stem(n, left_side, linefmt='b-', markerfmt='bo',
basefmt='r-')
plt.title(rf'$T(\alpha x)$ untuk $\alpha = \{\alpha\}$',
usetex=False)
plt.grid(True)

plt.subplot(3, 2, 2*i + 2)
plt.stem(n, right_side, linefmt='g-', markerfmt='go',
basefmt='r-')
plt.title(rf'$\alpha T(x)$ untuk $\alpha = \{\alpha\}$',
usetex=False)
plt.grid(True)

plt.tight_layout()
plt.show()
print("--- Kesimpulan Uji Homogenitas  $T(x) = 2x$  ---")
alpha_for_check = alphas[0] # Using 0.5 for the check
is_homogeneous = np.allclose(T(alpha_for_check * x_n),
alpha_for_check * T(x_n))
print(f"Apakah  $T(\alpha * x) == \alpha * T(x)$  untuk
 $\alpha=\{\alpha_{for\_check}\}$  {is_homogeneous}")
Output :

```



```

--- Kesimpulan Uji Homogenitas  $T(x) = 2x$  ---
Apakah  $T(\alpha * x) == \alpha * T(x)$  untuk  $\alpha=0.5$  True

```

B. Uji Additivitas

```
x1 = np.sin(0.5 * np.pi * n)
x2 = np.where(n >= 5, 1.0, 0.0)
left_add = T(x1 + x2)
right_add = T(x1) + T(x2)

difference = left_add - right_add

plt.figure(figsize=(15, 8))

plt.subplot(2, 3, 1)
plt.stem(n, x1, linefmt='c-')
plt.title('Input 1: x1[n]')
plt.grid(True)

plt.subplot(2, 3, 2)
plt.stem(n, x2, linefmt='y-')
plt.title('Input 2: x2[n]')
plt.grid(True)

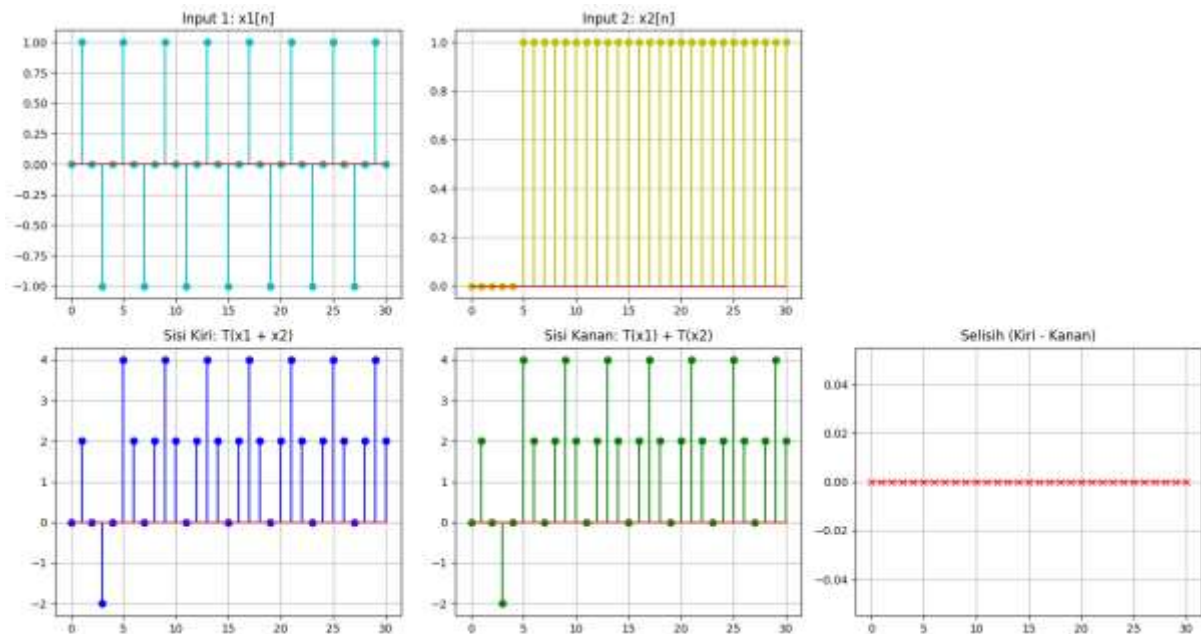
plt.subplot(2, 3, 4)
plt.stem(n, left_add, linefmt='b-', markerfmt='bo')
plt.title('Sisi Kiri: T(x1 + x2)')
plt.grid(True)

plt.subplot(2, 3, 5)
plt.stem(n, right_add, linefmt='g-', markerfmt='go')
plt.title('Sisi Kanan: T(x1) + T(x2)')
plt.grid(True)

plt.subplot(2, 3, 6)
plt.stem(n, difference, linefmt='r-', markerfmt='rx')
plt.title('Selisih (Kiri - Kanan)')
plt.grid(True)

plt.tight_layout()
plt.show()
print("--- Kesimpulan Uji Additivitas T(x) = 2x ---")
is_additive = np.allclose(left_add, right_add)
print(f"Apakah T(x1 + x2) == T(x1) + T(x2) {is_additive}")
```

Output :



--- Kesimpulan Uji Additivitas $T(x) = 2x$ ---
 Apakah $T(x1 + x2) == T(x1) + T(x2)$ True

C. Perbandingan Sistem Linier dan Non-Linier

```
def T2(x):
    return x ** 2

alpha_test = 3.0
homo_left_t2 = T2(alpha_test * x1)
homo_right_t2 = alpha_test * T2(x1)
is_homo_t2 = np.allclose(homo_left_t2, homo_right_t2)

add_left_t2 = T2(x1 + x2)
add_right_t2 = T2(x1) + T2(x2)
is_add_t2 = np.allclose(add_left_t2, add_right_t2)

print(f"--- Uji Sistem T2(x) = x^2 ---")
print(f"Memenuhi Homogenitas? {is_homo_t2}")
print(f"Memenuhi Additivitas? {is_add_t2}")
```

Output :

```
--- Uji Sistem T2(x) = x^2 ---
Memenuhi Homogenitas? False
Memenuhi Additivitas? False
```

Sistem	Homogenitas	Additivitas	Linier / Tidak Linier	Alasan
T1 (x)=2x	Ya	Ya	Linier	Memenuhi kedua syarat utama sistem linier (homogenitas dan additivitas) karena merupakan fungsi berderajat satu.

$T_2(x)=x^2$	Tidak	Tidak	Tidak Linier	Operasi kuadratik merusak sifat penskalaan (α) dan penjumlahan komponen. $(x_1+x_2)^2 \neq x_1^2+x_2^2$, melainkan menghasilkan komponen tambahan $2x_1x_2$.
--------------	-------	-------	--------------	---

Jawaban pertanyaan analisis :

- Ya, sistem $T_1(x) = 2x$ terbukti merupakan sistem linier. Berdasarkan hasil simulasi komputasi yang sudah kamu jalankan sebelumnya:
 - Uji Homogenitas: Menghasilkan nilai `True`. Grafik untuk sisi kiri $T(\alpha x)$ dan sisi kanan $\alpha T(x)$ menunjukkan bentuk dan nilai amplitudo yang persis sama di setiap variasi nilai.
 - Uji Additivitas: Menghasilkan nilai `True`. Grafik selisih antara respons penjumlahan input $T(x_1 + x_2)$ dengan penjumlahan masing-masing respons $T(x_1) + T(x_2)$ bernilai tepat 0 di sepanjang sumbu waktu.

- Sistem $T_2(x) = x^2$ gagal karena operasi pangkat dua (kuadratik) merusak sifat penskalaan skalar dan penjumlahan independen komponen sinyal. Penjelasan matematisnya adalah:

- Kegagalan Homogenitas

$$\text{Sisi Kiri: } T(\alpha x) = (\alpha x)^2 = \alpha^2 x^2$$

$$\text{Sisi Kanan: } \alpha T(x) = \alpha \cdot (x^2) = \alpha x^2$$

Karena $\alpha^2 x^2 \neq \alpha x^2$ (untuk nilai $\alpha \neq 1$ atau 0), aturan penskalaan menjadi rusak.

- Kegagalan Additivitas

$$\text{Sisi Kiri: } T(x_1 + x_2) = (x_1 + x_2)^2 = x_1^2 + 2x_1x_2 + x_2^2$$

$$\text{Sisi Kanan: } T(x_1) + T(x_2) = x_1^2 + x_2^2$$

Munculnya suku tambahan ($2x_1x_2$) di sisi kiri membuat kedua ruas tidak sama, sehingga sistem gagal bersifat aditif.

- Bedanya yaitu :

- Sistem Linier: Mematuhi prinsip superposisi. Sistem ini sangat konsisten dan dapat diprediksi karena tidak akan pernah menciptakan komponen frekuensi baru yang tidak ada pada sinyal masukan aslinya.
 - Sistem Non-Linier: Melanggar prinsip superposisi. Operasi non-linier (seperti kuadrat, akar, atau pembatasan nilai/clipping) akan merusak bentuk asli sinyal dan memunculkan komponen frekuensi baru (distorsi/harmonik) yang tidak dimiliki oleh sinyal input aslinya.
- Variasi nilai α (seperti bilangan bulat besar, pecahan, maupun bilangan negatif) diperlukan untuk memastikan bahwa sifat homogenitas sistem berlaku secara universal dan konsisten, bukan hanya kebetulan pada angka tertentu. Sebagai contoh, jika kita hanya menguji sistem $T(x) = x^2$ menggunakan nilai $\alpha = 1$ atau $\alpha = 0$, persamaannya akan tampak sama dan menghasilkan output `True`. Dengan memberikan variasi nilai pengali yang beragam, kita dapat menguji ketangguhan hukum linearitas sistem secara menyeluruh.

5. BAGIAN D : ANALISIS HOTS

A. Analisis Konseptual

- 1) Sistem linier didasarkan pada prinsip superposisi, yang menyatakan bahwa jika sebuah kombinasi linear dari beberapa sinyal dimasukkan ke dalam sistem, maka respons (output) sistem harus sama dengan kombinasi linear dari masing-masing respons individunya.
- 2) Karena tidak semua operasi matematika mempertahankan sifat proporsionalitas. Sebuah operasi hanya linier jika ia mematuhi syarat homogenitas dan aditivitas
- 3) Distorsinya adalah efek utamanya. Frekuensi harmonik baru yang sama sekali tidak ada pada sinyal asli akan diciptakan oleh sistem non-linier dengan merusak bentuk asli gelombang sinyal. Ini terdengar seperti suara yang pecah atau kasar dalam suara. Ini dapat berupa warna yang terlalu kontras, putihan, atau detail yang hilang pada gambar.
- 4) Operasi Shift (Penggeseran): Sangat penting untuk sinkronisasi waktu (time alignment). Sedangkan Operasi Amplify (Amplifikasi): Sangat penting untuk normalisasi tingkat energi.

B. Analisis Aplikasi Nyata

Studi Kasus : Image Blending

- 1) Masalah nyata yang ingin diselesaikan adalah menggabungkan dua informasi visual yang berbeda ke dalam satu frame gambar, seperti menambahkan watermark (logo transparan) pada gambar, membuat efek double exposure, atau menggabungkan eksposur gambar (HDR), semuanya tanpa menghasilkan silau putih polos atau kerusakan pada hasil akhir gambar.
- 2) Operasi dasar yang digunakan adalah kombinasi amplifikasi skalar dan penjumlahan citra
- 3) Karena jika dua gambar langsung ($\text{Image1} + \text{Image2}$) dijumlahkan begitu saja, nilai pikselnya akan menumpuk dan sering melampaui angka 255, sehingga gambar terkena clipping menjadi putih polos. Untuk memastikan bahwa nilai total penjumlahan piksel tetap berada di rentang 0-255 yang aman, kita menambahkan pembobotan persentase, yang diubah dengan nilai alpha di antara 0 dan 1.
- 4) Ya, bersifat linier. Karena persamaan blending hanya melibatkan perkalian dengan konstanta skalar (α) dan penjumlahan fungsi derajat satu.
- 5) Kelebihannya, sangat ringan, cepat secara komputasi dan mudah diimplementasikan secara real-time. Keterbatasannya, sistem ini “buta” terhadap konteks bentuk.

C. Skenario Pengambilan Keputusan

- 1) Skenario 1 : Image Blending dengan ukuran citra berbeda

Untuk memulai, kita harus menggunakan teknik interpolasi spasial (seperti bilinear atau bikubik) untuk memperbesar salah satu atau kedua gambar untuk memastikan bahwa tinggi dan lebar mereka sama persis. Ini sangat penting karena gambar diwakili secara digital dalam bentuk matriks dua dimensi. Proses penjumlahan gambar menghitung intensitas piksel setiap elemen, atau setiap elemen demi elemen yang berada pada koordinat yang sama. Hukum dasar aljabar linier menyatakan bahwa dua matriks hanya dapat dijumlahkan jika dan hanya jika keduanya memiliki ordo atau dimensi yang sama. Jika dimensinya berbeda, komputer akan mengalami error dan tidak dapat menemukan operasi matematika tersebut.

- 2) Skenario 2 : Sinyal audio terlalu kecil

Operasi yang dilakukan adalah amplifikasi (perkalian skalar), di mana setiap sampel data amplitudo dari sinyal audio tersebut dikalikan dengan sebuah nilai konstanta atau faktor pengali $\alpha > 1$. Risiko teknis terbesar jika nilai α terlalu besar adalah terjadinya Audio Clipping (Distorsi Amplitudo).

3) Skenario 3 : Deteksi Objek dengan posisi bervariasi

Operasi penggeseran (translasi) membantu meningkatkan performa model AI dengan cara memindahkan posisi objek target ke berbagai koordinat spasial yang berbeda (ke atas, bawah, kanan, atau kiri) di dalam bingkai gambar. Dalam bidang Machine Learning dan Computer Vision, Teknik ini dikenal dengan istilah Augmentasi Data.

4) Skenario 4 : Urutan subsistem cascade yang menghasilkan output berbeda

Sistem keseluruhan tersebut tidak dapat dikategorikan sebagai sistem linier. Fakta bahwa perubahan urutan subsistem menghasilkan output yang berbeda menjadi bukti valid bahwa prinsip superposisi telah dilanggar, yang menandakan bahwa minimal salah satu atau kedua subsistem di dalam rangkaian *cascade* tersebut memiliki karakteristik non-linier. Sifat non-linier inilah yang menyebabkan urutan operasi menjadi sensitif dan memengaruhi hasil akhir perhitungan matriks atau sinyalnya.

Jawaban pertanyaan refleksi pribadi :

1. Bagi saya, amplifikasi pada sinyal 1D. Secara matematis sangat lugas, saya hanya perlu mengalikan nilai amplitudo dengan sebuah konstanta untuk membesarkan atau mengecilkan sinyal secara proporsional
2. Penjumlahan citra 2D (*Image Blending*). Saya harus memastikan dimensi/resolusi kedua matriks gambar sama persis terlebih dahulu, lalu berhati-hati mengelola nilainya (misalnya dengan *clipping*) agar tidak melebihi batas 255 (*overflow*) yang bisa merusak gambar.
3. Pada sinyal 1D, operasi hanya dilakukan pada satu sumbu variabel (seperti waktu). Sedangkan citra 2D direpresentasikan dalam bentuk matriks (baris dan kolom), sehingga operasinya harus sangat mematuhi aturan aljabar linier seperti kesamaan dimensi (ordo) matriks.
4. Saya jadi paham bahwa sistem linier harus mutlak memenuhi prinsip superposisi (homogenitas dan additivitas). Persamaan yang terlihat simpel seperti $T(x) = x^2$ ternyata non-linier karena saat diuji akan merusak proporsi sinyal dan memunculkan distorsi.
5. Konsep normalisasi (*clipping*) dan translasi (*shift*). Normalisasi penting untuk mencegah distorsi ekstrem seperti suara pecah atau foto *whiteout*. Sedangkan translasi sangat berguna untuk augmentasi data AI agar sistem lebih tangguh mendeteksi posisi objek yang berbeda-beda.